

Rapport J05

SQL DML, filtrage, NULL et transactions

Plan d'exécution SQL/Data - Semaine 01

1. Identification

- Parcours : Plan d'exécution SQL/Data - 16 semaines
- Semaine : 01
- Jour : J05
- Base de données : sql_training
- SGBD : PostgreSQL 17.5
- Environnement : Windows 11 / PowerShell / psql / VS Code / Git
- Table de travail : raw.training_events
- Durée prévue : 90 min
- Durée réelle : 93 min
- Écart : +3 min (+3,3 %)

2. Objectifs

J05 avait pour objectif de passer de la création des structures PostgreSQL à la manipulation contrôlée des données :

1. utiliser SELECT et sélectionner explicitement les colonnes ;
2. employer des alias et des expressions calculées ;
3. filtrer avec WHERE, AND, OR et BETWEEN ;
4. comprendre NULL et la logique SQL à trois valeurs ;
5. trier avec ORDER BY, DESC et NULLS LAST ;
6. limiter les résultats avec LIMIT ;
7. effectuer un UPDATE contrôlé ;
8. tester un DELETE dans une transaction et l'annuler avec ROLLBACK ;
9. produire un script SQL reproductible ;
10. exécuter le script depuis PowerShell avec psql -f.

3. État de départ

La table raw.training_events, créée en J04, comprend :

```
event_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
event_type VARCHAR(50) NOT NULL,  
source_system VARCHAR(100) NOT NULL,  
event_value NUMERIC(12,2),  
event_timestamp TIMESTAMPTZ NOT NULL,  
ingested_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
```

Au début de J05, elle contenait deux événements issus de J04.

4. Enrichissement du jeu de données

Cinq événements supplémentaires ont été insérés. Après insertion, la table contenait 7 lignes.

Les identifiants observés étaient : 1, 3, 4, 5, 6, 7, 8.

Le trou dans la numérotation a permis de confirmer qu'une colonne IDENTITY ne garantit pas une séquence sans interruption : une tentative d'insertion échouée peut consommer une valeur.

5. SELECT, projection, alias et expressions

Une projection explicite a été utilisée :

```
SELECT
    event_id,
    event_type,
    event_value
FROM raw.training_events
ORDER BY event_id;
```

Des alias et une expression calculée ont ensuite été testés :

```
SELECT
    event_id AS id,
    event_type AS type_evenement,
    event_value AS valeur,
    event_value * 2 AS valeur_double
FROM raw.training_events
ORDER BY event_id;
```

Bonne pratique retenue : préférer une liste explicite de colonnes à SELECT * lorsque les colonnes nécessaires sont connues.

6. Filtrage avec WHERE, AND et OR

Exemple de filtre :

```
SELECT
    event_id,
    event_type,
    source_system,
    event_value
FROM raw.training_events
WHERE source_system = 'microgrid_poc'
ORDER BY event_id;
```

Résultat observé : 3 lignes.

Exemple avec plusieurs conditions :

```
WHERE source_system = 'microgrid_poc'
AND event_value > 20
```

Résultat observé : 2 lignes.

Un filtre avec OR a également été utilisé pour sélectionner plusieurs types d'événements.

7. NULL et logique SQL à trois valeurs

Les expressions suivantes ont été étudiées :

```
event_value IS NULL
event_value IS NOT NULL
event_value = NULL
event_value > 100
```

Concept retenu : NULL représente une valeur inconnue ou absente. Une comparaison telle que event_value = NULL ne donne pas un booléen classique exploitable comme attendu ; elle relève de la logique SQL à trois valeurs et produit UNKNOWN.

Les formes correctes sont :

```
event_value IS NULL
event_value IS NOT NULL
```

Après mise à jour de machine_alarm, aucune ligne ne contenait plus event_value IS NULL.

8. ORDER BY, DESC, NULLS LAST et LIMIT

Exemple :

```
SELECT
    event_id,
    event_type,
    event_value
FROM raw.training_events
ORDER BY event_value DESC NULLS LAST
LIMIT 3;
```

ORDER BY contrôle l'ordre, DESC impose l'ordre décroissant, NULLS LAST place les valeurs NULL à la fin et LIMIT réduit le nombre de lignes retournées.

9. UPDATE contrôlé

La ligne machine_alarm / tomra_training a d'abord été identifiée avec un SELECT, puis modifiée :

```
UPDATE raw.training_events
SET event_value = 1.00
WHERE event_type = 'machine_alarm'
AND source_system = 'tomra_training';
```

PostgreSQL a retourné UPDATE 1.

La requête de contrôle a confirmé :

- event_id = 7
- event_type = machine_alarm
- source_system = tomra_training
- event_value = 1.00

Bonne pratique retenue : exécuter d'abord un SELECT avec le même prédicat qu'un futur UPDATE ou DELETE.

10. Transaction DELETE / ROLLBACK

Une transaction explicite a été ouverte avec :

```
BEGIN;
```

La ligne current / microgrid_poc a été supprimée temporairement :

```
DELETE FROM raw.training_events
WHERE event_type = 'current'
AND source_system = 'microgrid_poc';
```

Pendant la transaction :

- la ligne n'était plus visible ;
- le nombre de lignes était passé de 7 à 6 ;
- l'invite psql affichait sql_training=#.

La transaction a été annulée :

```
ROLLBACK;
```

Après ROLLBACK, la ligne a été restaurée et COUNT(*) est revenu à 7.

11. Challenge autonome

Cinq exercices ont été réalisés en prévoyant d'abord le résultat.

Exercice	Prévision	Résultat
source_system = 'sensor_gateway'	2	2
event_value > 25	4	4
temperature OR voltage	3	3
event_value BETWEEN 20 AND 250	4	4
event_value IS NULL	0	0

Score : 5/5 - 100 %.

Les cinq prévisions ont été confirmées par PostgreSQL.

12. Script SQL reproductible

Le fichier suivant a été créé :

documentation/semaine-01/jour-05/code/J05_dml_queries_and_transactions.sql

Les requêtes de lecture y sont exécutables directement. Les exemples UPDATE et DELETE ont été placés dans un bloc de commentaires afin d'éviter une modification accidentelle lors d'une exécution complète.

Le script a été exécuté depuis PowerShell :

```
psql -U postgres -d sql_training -f ".\documentation\semaine-01\jour-05\code\J05_dml_queries_and_transactions.sql"
```

Une validation indépendante a ensuite été effectuée :

```
psql -U postgres -d sql_training -c "SELECT COUNT(*) AS total_events FROM raw.training_events;"
```

Résultat final : total_events = 7.

Le script s'est donc exécuté sans altérer accidentellement le jeu de données.

13. Captures principales

- J05-08_delete-rollback.png
- J05-09_sql-autonomous-challenge.png
- J05-10_vscode-dml-queries-and-transactions-script.png
- J05-11_powershell-sql-script-execution.png

Les captures antérieures de J05 documentent également les étapes de lecture, filtrage, gestion de NULL, tri et mise à jour.

14. Erreurs, difficultés et résolutions

14.1 Correction d'une requête multiligne dans psql

Une erreur de saisie pendant une requête multiligne a conduit à revoir le comportement du buffer psql.

Leçon : lorsqu'une requête interactive devient difficile à corriger, annuler le buffer puis ressaisir proprement la commande peut être plus sûr que poursuivre une requête incorrecte.

14.2 Comparaison avec NULL

La différence entre event_value = NULL et event_value IS NULL a été clarifiée.

Leçon : utiliser IS NULL ou IS NOT NULL pour tester l'absence de valeur.

14.3 Trou dans l'IDENTITY

Une insertion ayant échoué a consommé une valeur de l'identité.

Leçon : une clé IDENTITY garantit la génération d'identifiants, mais pas une numérotation continue sans trous.

15. Validation des compétences

- SELECT et projection : acquis
- alias et expressions : acquis
- WHERE : acquis
- AND / OR : acquis
- BETWEEN : acquis
- NULL : acquis
- ORDER BY / DESC / NULLS LAST : acquis
- LIMIT : acquis
- UPDATE contrôlé : exercé et validé
- transaction explicite : exercée et validée
- DELETE sécurisé par transaction : validé
- ROLLBACK : validé
- script SQL reproductible : validé
- exécution psql -f : validée

16. KPI J05

KPI	Résultat
Durée prévue	90 min
Durée réelle	93 min
Écart	+3 min
Écart relatif	+3,3 %

KPI	Résultat
Challenge autonome	5/5
Taux de réussite	100 %
Lignes finales dans la table	7
Modification accidentelle lors du test reproductible	0
Script SQL produit	1
Transaction avec ROLLBACK validée	Oui

17. Potentiel de réutilisation pédagogique

Sujet : SQL pratique avec PostgreSQL - lire, filtrer et modifier les données sans danger.

Public : débutants à intermédiaires SQL.

Tutoriel court : 8 à 12 minutes.

Module de formation : 45 à 60 minutes.

Démonstrations réutilisables :

- SELECT et projection ;
- WHERE, AND, OR et BETWEEN ;
- NULL et logique à trois valeurs ;
- ORDER BY et LIMIT ;
- UPDATE sécurisé ;
- DELETE dans une transaction ;
- ROLLBACK ;
- exécution d'un fichier SQL avec `psql -f`.

18. Conclusion

J05 a fait évoluer le parcours de la création de structures PostgreSQL vers la manipulation effective des données.

Trois pratiques importantes ont été consolidées :

1. prévoir le résultat avant d'exécuter une requête ;
2. sécuriser les modifications de données ;
3. transformer les commandes interactives en scripts reproductibles.

Le challenge autonome a été réussi à 100 %.

La séance prévue pour 90 minutes a duré 93 minutes, soit un écart de +3 minutes (+3,3 %).

19. Prochaines étapes

Pour clôturer définitivement J05 :

1. ajouter les rapports MD, DOCX et PDF au dossier J05 ;
2. effectuer le staging Git ;
3. contrôler le contenu du staging ;
4. créer le commit J05 ;
5. pousser vers GitHub ;
6. vérifier nothing to commit, working tree clean.